

Respuesta de actividad 1.1 y 1.2

Por Juan Manzur
De matricula: 21910516924

Actividad 1: relaciones a partir del enunciado

1. ¿Qué relaciones deberían incorporarse al diagrama? Indique las clases involucradas, el tipo de relación y justifique su elección.

Respuesta: Por lo menos deberían agregarse las siguientes tres relaciones:

- **Prestamo — Socio** como asociación, ya que “*cada préstamo guarda un socio.*”, una clara alusión a una asociación, pues la clase Prestamo deberá guardar en su composición a la clase Socio de manera estructural a través del historial.
- **Prestamo — Libro** como asociación, se repite el mismo caso que la relación anterior ya que “*cada préstamo guarda un libro.*”, haciendo que Prestamo guarde a Libro en su interior a través del historial de la misma manera que Socio.
- **Prestamo - -> Impresora** como dependencia, ya que la clase Impresora es usada para imprimir el comprobante de la clase Prestamo, pero el enunciado aclara que “*El préstamo no guarda la impresora.*”, por ende su relación es temporal, solo se relacionan a través de la impresión.

2. Para cada asociación propuesta, indique sus multiplicidades y explique qué significan en el caso.

Respuesta: Para cada asociación propuesta en la pregunta anterior, la relación de sus multiplicidades:

- **Prestamo — Socio** 0..* : 1, ya que cada Socio puede tener relación con muchos préstamos en el historial, pero un Prestamo solo tiene guardado un Socio.
- **Prestamo — Libro** 0..* : 1, ya que cada Libro puede tener relación con muchos préstamos en el historial, pero un Prestamo solo tiene guardado un Libro.

3. ¿Bastaría representar todas las relaciones como dependencias para describir el caso? Justifique con el enunciado.

Respuesta: No. Ya que el mismo enunciado pide persistencia para el caso al describir *“Una biblioteca registra los préstamos de libros a sus socios.”* y también al pedir como requisito *“El préstamo se conserva en el historial.”*, al hacer alusión de mantener un registro que se debe conservar, no es posible lograrlo únicamente a través de dependencias, debido a que las dependencias trabajan como relaciones temporales, por ende no serían capaces de llevar un registro persistente.

4. ¿Qué objetos debería guardar **“Prestamo”** como atributos? Justifique.

Respuesta: Los atributos de tipo objeto que deberían ser guardados en la clase Prestamo son:

- objeto socio tipo **Socio**: debido que la clase Socio mantiene una relación de asociación con la clase Prestamo, y ya se dejó en claro previamente que esta clase deben persistir guardado dentro de Prestamo en el historial.
- objeto libro tipo **Libro**: debido que la clase Libro mantiene una relación de asociación con la clase Prestamo, y ya se dejó en claro previamente que esta clase deben persistir guardado dentro de Prestamo en el historial.

Nota: Teniendo en cuenta la cita del enunciado *“el préstamo se marca como devuelto.”* y tomando como referencia el diagrama de clase de Prestamo, podríamos incluir un tercer atributo denominado devuelto de tipo **boolean**, que marque el estado del Prestamo en el historial, pero este atributo no contaría para la respuesta, ya que se piden atributos de tipo “Objetos” no cualquier tipo de atributos.

5. ¿Qué parámetros y tipos propondría para **“devolver”**? Explique para qué sirve cada uno.

Respuesta: Conceptualmente, el método denominado como devolver(...), sería un método usado únicamente al momento en el que un socio devuelve un libro prestado, por ende podría llevar los siguientes parámetros con sus respectivos tipos:

- int diasAtrasados, ya que si pide que haya un conteo de los días atrasados para agregar una multa en función de estos, siendo de tipo int, ya que el conteo debe ser de números enteros al contar exclusivamente los días.
- Impresora impresora, ya que el método devolver(...), debe generar el comprobante y enviarlo a imprimir, que se menciona en el enunciado de forma *“El comprobante se imprime con la impresora disponible en ese momento. El préstamo no guarda la impresora”*, haciendo alusión que la impresora de tipo Impresora, debe ser usada como parámetro y no como atributo.

Actividad 2: relaciones en el código

1. ¿Dónde se reflejan las asociaciones y dependencias en el código? Señale los atributos o métodos, las clases involucradas y justifique cada relación.

Respuesta: De las asociaciones y dependencia previstas se ven reflejadas en el código:

- **Prestamo — Socio:** Esta relación es una asociación, se refleja en el atributo de la clase Prestamo. El constructor recibe un Socio y lo asigna a ese atributo. Luego Prestamo usa el método get de Socio para construir el comprobante. La clase Socio encapsula su nombre y lo expone:
 - En la clase Prestamo “ *private Socio socio;* ”
 - En la clase Prestamo “ *public Prestamo(Socio socio, Libro libro)* ”
 - En la clase Socio “ *public String getNombre()* ”
 - En la clase Prestamo “ “*Socio:* ” + *socio.getNombre()* ”
- **Prestamo — Libro:** Esta relación es una asociación, se refleja en el atributo de la clase Prestamo. El constructor recibe un Libro y lo asigna a ese atributo. Luego Prestamo usa el método get de Libro para construir el comprobante. La clase Socio encapsula su nombre y lo expone:
 - En la clase Prestamo “ *private Libro libro;* ”
 - En la clase Prestamo “ *public Prestamo(Socio socio, Libro libro)* ”
 - En la clase Libro “ *public String getTitulo()* ”
 - En la clase Prestamo “ “*\nLibro:* ” + *libro.getTitulo()* ”
- **Prestamo - - -> Impresora:** Esta relación es una dependencia, de modo que la clase Prestamo utiliza el objeto impresora de tipo Impresora únicamente como parámetro y para llamar el método imprimir, pero sin almacenar nada de la clase Impresora:
 - En la clase Prestamo “ *public void devolver(int diasAtraso, Impresora impresora)* ”
 - En la clase Prestamo “ *impresora.imprimir(comprobante);* ”

2. ¿Cuántas asociaciones y dependencias hay entre las cuatro clases?

Respuesta: Si contamos cada relación una única vez, hay dos asociaciones (**Prestamo — Socio** y **Prestamo — Libro**) y una dependencia (**Prestamo - - -> Impresora**) entre las cuatro clases.

3. ¿Existe una relación directa entre “**Socio**” y “**Libro**”? Justifique con el código.

Respuesta: No. No hay una relación directa entre las clases Socio y Libro, ya que en el código se evidencia que ninguna de estas clases almacena o usa métodos o atributos de la otra clase, solo tienen una relación indirecta al “juntarse” dentro de la clase Prestamo.

4. ¿La clase “**Prestamo**” aplica SRP? Justifique según sus razones para cambiar y proponga una separación si la considera necesaria.

Respuesta: No, no cumple con el principio de única responsabilidad (Single Responsibility Principle), ya que la clase Prestamo tiene las siguientes responsabilidades:

- 1) Manipula el estado del préstamo con los atributos **socio**, **libro** y **devuelto**.
- 2) Calcula el capital de la multa de un atraso en el método **devolver(...)**
- 3) Realiza el comprobante creando el texto en el método **devolver(...)**
- 4) Dirige la impresión del comprobante usando la clase **Impresora**, en el método **devolver(...)**.

Por ende sus **razones para cambiar** serían:

- Si cambia la regla de la multa.
- Si cambia el formato del comprobante.
- Si cambia la forma de imprimir.
- Si cambia la lógica del estado del préstamo

Un posible **arreglo de clases** que yo propondría, sería el siguiente:

- **Prestamo**: Clase modificada para únicamente guardar el estado de un préstamo (**socio**, **libro**, **devuelto**).
- **Impresora**: Clase mantenida para cumplir su única responsabilidad de **imprimir(...)**.
- **CalculaMulta**: Clase creada para calcular matemáticamente el capital de la **multa** de cada de cada préstamo según **diasAtraso**.
- **GeneraComprobante**: Clase creada para crear el texto de cada **comprobante**.
- **ServicioDevolucion**: Clase creada para coordinar el servicio de devolución, podría tener de parámetro **diasAtraso**, por ende, manipula el **Prestamo** para modificar **devuelto**, solicita la **multa** a **CalculaMulta**, solicita el **comprobante** a **GeneraComprobante** y solicita **imprimir(...)** a **Impresora**.